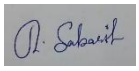
SIMATS ENGINEERING
SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
CHENNAI – 602105
Department of Computer Science and Engineering
ASSESSMENT TOOL 2 – SCHEMA ANALYSIS AND VIVA VOCE

Course Code:	**DSA05**	**Course Title:**	**Query Processing for Data Science With Real Time Applications**
CO Assessed:	**CO2**	**Assessment:**	**Tool 2 – Schema Analysis and Viva Voce**
Weightage:		**Total Marks:**	**20 Marks**
CO2 Statement: Use Python basics and SQL libraries to connect to databases SQLite, MySQL, PostgreSQL and perform CRUD operations like Create, Read, Update, Delete. (BL6)			
Student Name:	**SABARISH R**	**Reg. No.:**	**192324092**
Section / Batch:	**4th YEAR SLOT A**	**Date of Submission:**	**05-08-2026**
Faculty In charge:	**K. Veena Devi**	**Project Mode:**	**Individual Submission**

Code of Conduct

I SABARISH R (192324092) (Name / Reg No) certify that this submission is my original work and that I have adhered to all guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature: 

Identification of Entities and Attributes (4 Marks)	Understanding of Relationships (4 Marks)	Analysis of Keys and Constraints (4 Marks)	Detection of Design Issues (4 Marks)	Viva Voce (4 Marks)	Total (20 Marks)

CO2 Assessment Tool 2 – Schema Analysis and Viva Voce

1. University Course Registration System

SQL Schema and Query

```
CREATE TABLE Department (
  DepartmentID INT PRIMARY KEY,
  DepartmentName VARCHAR(50)
);

CREATE TABLE Faculty (
  FacultyID INT PRIMARY KEY,
  FacultyName VARCHAR(50)
);

CREATE TABLE Student (
  StudentID INT PRIMARY KEY,
  Name VARCHAR(50),
  DepartmentID INT,
  FOREIGN KEY (DepartmentID) REFERENCES Department(DepartmentID)
);

CREATE TABLE Course (
  CourseID INT PRIMARY KEY,
  CourseName VARCHAR(50),
  FacultyID INT,
  FOREIGN KEY (FacultyID) REFERENCES Faculty(FacultyID)
);

CREATE TABLE Enrollment (
  StudentID INT,
  CourseID INT,
  Semester VARCHAR(10),
  PRIMARY KEY (StudentID, CourseID),
  FOREIGN KEY (StudentID) REFERENCES Student(StudentID),
  FOREIGN KEY (CourseID) REFERENCES Course(CourseID)
);

-- Query: list every student with the courses they enrolled in
SELECT S.Name, C.CourseName, E.Semester
FROM Student S
JOIN Enrollment E ON S.StudentID = E.StudentID
JOIN Course C ON E.CourseID = C.CourseID;
```

Q1. Why is the Enrollment table necessary instead of storing CourseID directly in the Student table?

Because one student can join many courses, and one course can have many students. This is a many-to-many relationship. If we just put CourseID inside the Student table, each student row could only hold one course, so we could not show a student taking more than one course. The Enrollment table acts like a bridge/link table between Student and Course, and it can also store extra details like Semester for every enrollment.

Q2. If the university wants to track grades for each enrolled course, how would you modify the schema?

We just add a new column Grade in the Enrollment table itself, since each row already represents one student taking one course:

ALTER TABLE Enrollment ADD Grade VARCHAR(2);

So Enrollment becomes Enrollment(StudentID, CourseID, Semester, Grade).

2. Hospital Management System

SQL Schema and Query

```
CREATE TABLE Patient (
  PatientID INT PRIMARY KEY,
  Name VARCHAR(50),
  Age INT
);

CREATE TABLE Doctor (
  DoctorID INT PRIMARY KEY,
  DoctorName VARCHAR(50),
  Specialization VARCHAR(50)
);

CREATE TABLE Appointment (
  AppointmentID INT PRIMARY KEY,
  PatientID INT,
  DoctorID INT,
  Date DATE,
  FOREIGN KEY (PatientID) REFERENCES Patient(PatientID),
  FOREIGN KEY (DoctorID) REFERENCES Doctor(DoctorID)
);

CREATE TABLE Prescription (
  PrescriptionID INT PRIMARY KEY,
  AppointmentID INT,
  Medicine VARCHAR(50),
  FOREIGN KEY (AppointmentID) REFERENCES Appointment(AppointmentID)
);

-- Query: show which doctor gave which medicine to which patient
SELECT P.Name AS Patient, D.DoctorName, Pr.Medicine
FROM Appointment A
JOIN Patient P ON A.PatientID = P.PatientID
JOIN Doctor D ON A.DoctorID = D.DoctorID
JOIN Prescription Pr ON Pr.AppointmentID = A.AppointmentID;
```

Q1. What problems might occur if prescription details are stored directly in the Appointment table?

If we put the medicine column straight into the Appointment table, then one appointment row can only store one medicine name. If the doctor gives 2 or 3 medicines in the same appointment, we would need to repeat the whole appointment row again and again, which causes data redundancy (same data repeated) and makes updates harder (update anomaly).

Q2. How would the schema change if a prescription contains multiple medicines?

We split Medicine out into its own table connected to Prescription, so many medicine rows can belong to one prescription:

```
CREATE TABLE PrescriptionMedicine (
  PrescriptionID INT,
  MedicineName VARCHAR(50),
  Dosage VARCHAR(20),
  PRIMARY KEY (PrescriptionID, MedicineName),
  FOREIGN KEY (PrescriptionID) REFERENCES Prescription(PrescriptionID)
);
```

3. Online Shopping System

SQL Schema and Query

```
CREATE TABLE Customer (
  CustomerID INT PRIMARY KEY,
  Name VARCHAR(50)
);

CREATE TABLE Product (
  ProductID INT PRIMARY KEY,
  ProductName VARCHAR(50),
  Price DECIMAL(10,2)
);

CREATE TABLE Orders (
  OrderID INT PRIMARY KEY,
  CustomerID INT,
  OrderDate DATE,
  FOREIGN KEY (CustomerID) REFERENCES Customer(CustomerID)
);

CREATE TABLE OrderDetails (
  OrderID INT,
  ProductID INT,
  Quantity INT,
  PRIMARY KEY (OrderID, ProductID),
  FOREIGN KEY (OrderID) REFERENCES Orders(OrderID),
  FOREIGN KEY (ProductID) REFERENCES Product(ProductID)
);

-- Query: show every product ordered under each order
SELECT O.OrderID, P.ProductName, OD.Quantity
FROM Orders O
JOIN OrderDetails OD ON O.OrderID = OD.OrderID
JOIN Product P ON OD.ProductID = P.ProductID;
```

Q1. Why is the OrderDetails table required in this schema?

Because one order can contain many products, and one product can appear in many different orders. This is a many-to-many relationship. OrderDetails acts as the bridge table between Order and Product, and it also stores the Quantity of each product bought in that order.

Q2. What would happen if ProductID were stored directly in the Order table?

Then one order row could hold only one ProductID, so a customer buying many different products in a single order would not be possible without repeating the whole order row for every product, which causes data redundancy and makes it confusing to calculate total order amount.

4. Library Management System

SQL Schema and Query

```
CREATE TABLE Member (
  MemberID INT PRIMARY KEY,
  Name VARCHAR(50)
);

CREATE TABLE Book (
  BookID INT PRIMARY KEY,
  Title VARCHAR(50),
  Author VARCHAR(50)
);

CREATE TABLE Issue (
  MemberID INT,
  BookID INT,
  IssueDate DATE,
  ReturnDate DATE,
  PRIMARY KEY (MemberID, BookID, IssueDate),
  FOREIGN KEY (MemberID) REFERENCES Member(MemberID),
  FOREIGN KEY (BookID) REFERENCES Book(BookID)
);

-- Query: show which member issued which book and when
SELECT M.Name, B.Title, I.IssueDate, I.ReturnDate
FROM Issue I
JOIN Member M ON I.MemberID = M.MemberID
JOIN Book B ON I.BookID = B.BookID;
```

Q1. Explain the relationship between Member and Book through the Issue table.

Member and Book have a many-to-many relationship: one member can borrow many books, and one book can be borrowed by many different members (at different times). The Issue table connects them by storing MemberID, BookID, IssueDate and ReturnDate, so it works like a bridge table that records every borrowing event.

Q2. How would you modify the schema to track overdue fines?

Add a FineAmount column to the Issue table:

`ALTER TABLE Issue ADD FineAmount DECIMAL(10,2) DEFAULT 0;`

We can calculate it using a query that checks if ReturnDate is after the due date, for example:

SELECT MemberID, BookID, DATEDIFF(ReturnDate, IssueDate) - 14 AS DaysLate FROM Issue WHERE DATEDIFF(ReturnDate, IssueDate) > 14;

5. Banking System

SQL Schema and Query

```
CREATE TABLE Customer (
  CustomerID INT PRIMARY KEY,
  Name VARCHAR(50)
);

CREATE TABLE Account (
  AccountNo INT PRIMARY KEY,
  CustomerID INT,
  Balance DECIMAL(10,2),
  FOREIGN KEY (CustomerID) REFERENCES Customer(CustomerID)
);

CREATE TABLE Transactions (
  TransactionID INT PRIMARY KEY,
  AccountNo INT,
  Amount DECIMAL(10,2),
  Type VARCHAR(10),
  FOREIGN KEY (AccountNo) REFERENCES Account(AccountNo)
);

-- Query: show all transactions of every customer
SELECT C.Name, A.AccountNo, T.Amount, T.Type
FROM Customer C
JOIN Account A ON C.CustomerID = A.CustomerID
JOIN Transactions T ON A.AccountNo = T.AccountNo;
```

Q1. Why should transactions be stored separately from account details?

Because one account can have hundreds of transactions over time (deposits, withdrawals), but an account has only one balance value at a time. If we tried to store every transaction inside the Account table, we would need unlimited columns, which is not possible. Keeping Transaction as a separate table lets us store as many transactions as needed for each account without repeating account details.

Q2. How would you redesign the schema to support joint accounts?

Right now Account has only one CustomerID, so one account belongs to only one customer. For joint accounts (many customers sharing one account), we need a bridge table instead:

```
CREATE TABLE AccountHolder (
  AccountNo INT,
  CustomerID INT,
  PRIMARY KEY (AccountNo, CustomerID),
  FOREIGN KEY (AccountNo) REFERENCES Account(AccountNo),
  FOREIGN KEY (CustomerID) REFERENCES Customer(CustomerID)
);
```

This makes Customer and Account a many-to-many relationship, so many customers can be linked to the same account.

6. Employee Payroll System

SQL Schema and Query

```
CREATE TABLE Department (
  DepartmentID INT PRIMARY KEY,
  DepartmentName VARCHAR(50)
);

CREATE TABLE Employee (
  EmployeeID INT PRIMARY KEY,
  Name VARCHAR(50),
  DepartmentID INT,
  FOREIGN KEY (DepartmentID) REFERENCES Department(DepartmentID)
);

CREATE TABLE Payroll (
  PayrollID INT PRIMARY KEY,
  EmployeeID INT,
  Salary DECIMAL(10,2),
  Month VARCHAR(15),
  FOREIGN KEY (EmployeeID) REFERENCES Employee(EmployeeID)
);

-- Query: show employee, department and their salary for every month
SELECT E.Name, D.DepartmentName, P.Month, P.Salary
FROM Employee E
JOIN Department D ON E.DepartmentID = D.DepartmentID
JOIN Payroll P ON E.EmployeeID = P.EmployeeID;
```

Q1. Why is Department information separated into a different table?

Because many employees can belong to the same department. If we keep DepartmentName inside the Employee table itself, the same department name gets repeated for every employee in that department. Keeping Department as its own table and just linking with DepartmentID avoids this repetition (this is called normalization).

Q2. What redundancy issues would arise if department details were stored in Employee records?

The DepartmentName (and other department info) would be duplicated for every employee in that department, wasting storage. Also, if the department name changes, we would have to update it in every single employee row instead of just one row in the Department table, which can easily cause mistakes (update anomaly).

7. Hotel Reservation System

SQL Schema and Query

```
CREATE TABLE Customer (
  CustomerID INT PRIMARY KEY,
  Name VARCHAR(50)
);

CREATE TABLE Room (
  RoomID INT PRIMARY KEY,
```

```
  RoomType VARCHAR(20)
);

CREATE TABLE Reservation (
  ReservationID INT PRIMARY KEY,
  CustomerID INT,
  RoomID INT,
  CheckIn DATE,
  CheckOut DATE,
  FOREIGN KEY (CustomerID) REFERENCES Customer(CustomerID),
  FOREIGN KEY (RoomID) REFERENCES Room(RoomID)
);

-- Query: check if a room is already booked for the dates we want
SELECT * FROM Reservation
WHERE RoomID = 101
AND CheckIn < '2026-08-10' AND CheckOut > '2026-08-05';
```

Q1. Why should reservation details be maintained separately from room details?

Because one room gets booked by many different customers at many different times, so Room only needs to store fixed details like RoomType once. Reservation is separate so we can store many booking records (with CheckIn/CheckOut dates) for the same room without repeating the room's own details every time.

Q2. How would you prevent double booking of rooms using the schema?

Before inserting a new reservation, we run a query to check if the same RoomID already has a reservation that overlaps with the new CheckIn/CheckOut dates (like the query above). If any row is returned, the room is already booked in that period, so we reject the new booking. We can also add a trigger or application-level check to enforce this automatically.

8. Food Delivery Application

SQL Schema and Query

```
CREATE TABLE Customer (
  CustomerID INT PRIMARY KEY,
  Name VARCHAR(50)
);

CREATE TABLE Restaurant (
  RestaurantID INT PRIMARY KEY,
  Name VARCHAR(50)
);

CREATE TABLE Orders (
  OrderID INT PRIMARY KEY,
  CustomerID INT,
  RestaurantID INT,
  FOREIGN KEY (CustomerID) REFERENCES Customer(CustomerID),
  FOREIGN KEY (RestaurantID) REFERENCES Restaurant(RestaurantID)
);

CREATE TABLE DeliveryAgent (
  AgentID INT PRIMARY KEY,
```

```
  Name VARCHAR(50)
);

CREATE TABLE Delivery (
  OrderID INT,
  AgentID INT,
  PRIMARY KEY (OrderID, AgentID),
  FOREIGN KEY (OrderID) REFERENCES Orders(OrderID),
  FOREIGN KEY (AgentID) REFERENCES DeliveryAgent(AgentID)
);

-- Query: show which agent delivered which order
SELECT O.OrderID, C.Name AS Customer, DA.Name AS Agent
FROM Orders O
JOIN Customer C ON O.CustomerID = C.CustomerID
JOIN Delivery D ON O.OrderID = D.OrderID
JOIN DeliveryAgent DA ON D.AgentID = DA.AgentID;
```

Q1. Why is Delivery maintained as a separate table?

Delivery connects Order and DeliveryAgent, which is a different concern from the order itself (what was ordered, from which restaurant). Keeping it separate lets us update or track delivery status (like which agent is assigned) without touching the Order table, and it also makes it easy to later allow more than one agent per order.

Q2. How would the schema change if multiple delivery agents could handle a single order?

The schema I wrote already supports this because Delivery has a composite primary key (OrderID, AgentID), not just OrderID alone. So we can simply insert more than one row with the same OrderID but different AgentID, and both agents get linked to the same order. This makes Order and DeliveryAgent a many-to-many relationship.

9. Learning Management System (LMS)

SQL Schema and Query

```
CREATE TABLE Student (
  StudentID INT PRIMARY KEY,
  Name VARCHAR(50)
);

CREATE TABLE Course (
  CourseID INT PRIMARY KEY,
  CourseName VARCHAR(50)
);

CREATE TABLE Enrollment (
  StudentID INT,
  CourseID INT,
  PRIMARY KEY (StudentID, CourseID),
  FOREIGN KEY (StudentID) REFERENCES Student(StudentID),
  FOREIGN KEY (CourseID) REFERENCES Course(CourseID)
);

CREATE TABLE Assignment (
```

```
  AssignmentID INT PRIMARY KEY,
  CourseID INT,
  FOREIGN KEY (CourseID) REFERENCES Course(CourseID)
);

-- Query: list students with the courses they are enrolled in
SELECT S.Name, C.CourseName
FROM Student S
JOIN Enrollment E ON S.StudentID = E.StudentID
JOIN Course C ON E.CourseID = C.CourseID;
```

Q1. Why is Enrollment considered a bridge table?

Because Student and Course have a many-to-many relationship — one student can join many courses, and one course can have many students. Enrollment just holds the foreign keys of both tables (StudentID and CourseID) and links them together, without holding any real data of its own, which is exactly what a bridge (junction) table does.

Q2. How would you modify the schema to record assignment grades?

We create a new table that links Assignment and Student, since each student gets a separate grade for each assignment:

```
CREATE TABLE AssignmentGrade (
  AssignmentID INT,
  StudentID INT,
  Grade VARCHAR(2),
  PRIMARY KEY (AssignmentID, StudentID),
  FOREIGN KEY (AssignmentID) REFERENCES Assignment(AssignmentID),
  FOREIGN KEY (StudentID) REFERENCES Student(StudentID)
);
```

10. Bus Ticket Reservation System

SQL Schema and Query

```
CREATE TABLE Passenger (
  PassengerID INT PRIMARY KEY,
  Name VARCHAR(50)
);

CREATE TABLE Bus (
  BusID INT PRIMARY KEY,
  Route VARCHAR(50)
);

CREATE TABLE Booking (
  BookingID INT PRIMARY KEY,
  PassengerID INT,
  BusID INT,
  TravelDate DATE,
  FOREIGN KEY (PassengerID) REFERENCES Passenger(PassengerID),
  FOREIGN KEY (BusID) REFERENCES Bus(BusID)
);

-- Query: show every booking with passenger name and bus route
```

```
SELECT P.Name, B.Route, Bk.TravelDate
FROM Booking Bk
JOIN Passenger P ON Bk.PassengerID = P.PassengerID
JOIN Bus B ON Bk.BusID = B.BusID;
```

Q1. Why should booking details be stored separately from passenger information?

Because one passenger can make many bookings (different buses, different dates), but a passenger's own details (Name) stay the same every time. If we stored booking info inside the Passenger table, we would have to repeat the passenger's details for every booking, causing data redundancy. Keeping Booking separate avoids this.

Q2. If a booking contains multiple passengers, how would you redesign the schema?

Remove PassengerID from the Booking table and instead create a bridge table, since one booking can now have many passengers and one passenger can be part of many bookings:

```
CREATE TABLE BookingPassenger (
  BookingID INT,
  PassengerID INT,
  PRIMARY KEY (BookingID, PassengerID),
  FOREIGN KEY (BookingID) REFERENCES Booking(BookingID),
  FOREIGN KEY (PassengerID) REFERENCES Passenger(PassengerID)
);
```

This makes Booking and Passenger a many-to-many relationship.